Homework #4

CSE 546: Machine Learning
Tyler Shakibai

Collaborators: Jake Murray, Peter Sushko

Conceptual Questions

A1. The answers to these questions should be answerable without referring to external materials. Briefly justify your answers with a few words.

a. *[2 points]* True or False: Given a data matrix $X \in \mathbb{R}^{n \times d}$ where d is much smaller than n and $k = \text{rank}(X)$, if we project our data onto a k-dimensional subspace using PCA, our projection will have zero reconstruction error (in other words, we find a perfect representation of our data, with no information loss).

True. The rank of X is the number of non-zero eigenvalues. The other entries in X can be represented as a linear combination of k basis vectors. Thus, if we project our data onto a k-dimensional subspace using PCA, the reconstruction error will be zero.

b. *[2 points]* True or False: Suppose that an $n \times n$ matrix X has a singular value decomposition of $USV^\top$, where S is a diagonal $n \times n$ matrix. Then, the rows of V are equal to the eigenvectors of $X^\top X$.

False. The columns of V are equal to the eigenvectors of $X^\top X$.

c. *[2 points]* True or False: choosing k to minimize the k-means objective (see Equation (1) below) is a good way to find meaningful clusters.

False. If $k = n$, the objective function is always the global minimum and it does not provide any meaningful interpretations of the data.

d. *[2 points]* True or False: The singular value decomposition of a matrix is unique.

False. The singular values are unique. However, the columns of U and V in the decomposition can differ.

e. *[2 points]* True or False: The rank of a square matrix equals the number of its unique nonzero eigenvalues.

False. The rank of a square matrix equals the number of its non-zero eigenvalues. These non-zero eigenvalues can be repeated.

Think before you train

A2. **The first part of this problem (part a)** explores how you would apply machine learning theory and techniques to a real-world problem. There is one scenario detailing a setting, a dataset, and a specific result we hope to achieve. Your job is to describe how you would handle the scenario with the tools we've learned in this class. Your response should include:

(1) any pre-processing steps you would take (i.e., data acquisition and processing),

(2) the specific machine learning pipeline you would use (i.e., algorithms and techniques learned in this class),

(3) how your setup acknowledges the constraints and achieves the desired result.

You should also aim to leverage some of the theory we have covered in this class. Some things to consider may be: the nature of the data (i.e., *How hard is it to learn? Do we need more data? Are the data sources good?*), the effectiveness of the pipeline (i.e., *How strong is the model when properly trained and tuned?*), and the time needed to effectively perform the pipeline.

a. *[10 points]* **Scenario: Disease Susceptibility Predictor**

- Setting: You are tasked by a research institute to create an algorithm that learns the factors that contribute most to acquiring a specific disease.
- Dataset: A rich dataset of personal demographic information, location information, risk factors, and whether a person has the disease or not.
- Result: The company wants a system that can determine how susceptible someone is to this disease when they enter in their own personal information. The pipeline should take limited amount of personal data from a new user and infer more detailed metrics about the person.

Since this is personal demographic information, we may encounter instances of incomplete data that need to be wrangled and cleansed of missing values in an appropriate way. Due to the tabular nature of the dataset, we can compartmentalize it as follows: we can make x_{train} consist of the demographic information, location information, and risk factors, and y_{train} be whether a person has the disease or not. We can one-hot encoding on our demographic data to ascribe numerical values to categories such as gender, age, ethnicity, income, etc. Since we have labels for both input and output this is clearly a supervised learning task. We can employ logistic regression for binary classification for this problem, then evaluate the binary classifier. We can also use simple neural networks, which might be more robust to redundant features should there be any and could better handle missing inputs. Our output would be a scalar quantity representing the susceptibility of this person to the disease as a probability.

The second part of this problem (parts b, c) focuses on exploring possible shortcomings of machine learning models, and what real-world implications might follow from ignoring these issues.

b. *[5 points]* Briefly describe (1) some potential shortcomings of your training process from the disease susceptibility predictor scenario above that may result in your algorithm having different accuracy on different populations, and (2) how you may modify your procedure to address these shortcomings.

In the training process for the aforementioned scenario, the issue of incomplete data was raised and would need to be filtered out or considered in an appropriate way. One approach would be to look for which inputs there is the least amount of data for. There could be a trend that individuals are unwilling to share certain demographic data. We could also search for the correlation between features we have data for. Risk factors such as aging, nutrition levels, infection, and exposure to toxic substances could be linked to location. Thus, some features that the algorithm associates with the disease may be redundant, which can lead to certain factors being disproportionately emphasized. The approach could also be modified to incorporate feature engineering, for example, reducing the dimensions of our data with principal component analysis. Another option could be to modify our goal and create different models for different populations and evaluate these models in comparison. This may give us different insights into the problem. In general, the task of analyzing health data should be approached in a thoughtful manner as it may have significant consequences relating to the health and privacy of many individuals.

c. *[5 points]* Recall in Homework 2 we trained models to predict crime rates using various features. It is important to note that **datasets describing crime have many shortcomings in describing the entire landscape of illegal behavior in a city, and that these shortcomings often fall disproportionately on minority communities**. Some of these shortcomings include that crimes are reported at different rates in different neighborhoods, that police respond differently to the same crime reported or observed in different neighborhoods, and that police spend more time patrolling in some neighborhoods than others. What real-world implications might follow from ignoring these issues?

Real-world implications that might follow from ignoring shortcomings in datasets describing crime include that certain neighborhoods may be disproportionately viewed as unsafe when consideration is not given to the fact that the way in which crimes are reported and in turn garner a police response is different depending on the neighborhood. These shortcomings in the dataset can then affect policy decisions about infrastructure with implications for schooling, real estate value, and what publicly-funded resources are available. Furthermore, the definition of a crime may vary depending on the neighborhood in question. Infractions such as speeding and loitering may be conflated with more significant offenses relating to drugs or violence. In practice, crime statistics and their shortcomings disproportionately affect minority communities and can enforce policies that have negative long-term impact.

k-means clustering

A3. Given a dataset $\mathbf{x}_1, ..., \mathbf{x}_n \in \mathbb{R}^d$ and an integer $1 \leq k \leq n$, recall the following k-means objective function

$$\min_{\pi_1,...,\pi_k} \sum_{i=1}^{k} \sum_{j \in \pi_i} \|\mathbf{x}_j - \mu_i\|_2^2 \,, \quad \mu_i = \frac{1}{|\pi_i|} \sum_{j \in \pi_i} \mathbf{x}_j \,. \tag{1}$$

Above, $\{\pi_i\}_{i=1}^k$ is a partition of $\{1, 2, ..., n\}$. The objective (1) is NP-hard[1] to find a global minimizer of. Nevertheless, Lloyd's algorithm (discussed in lecture) typically works well in practice.

a. *[5 points]* Implement Lloyd's algorithm for solving the k-means objective (1). Do not use any off-the-shelf implementations, such as those found in `scikit-learn`. Include your code in your submission.

```
@problem.tag("hw4-A")
def calculate_centers(
    data: np.ndarray, classifications: np.ndarray, num_centers: int
) -> np.ndarray:
    """
    Sub-routine of Lloyd's algorithm that calculates the centers given datapoints a
    num_centers is additionally provided for speed-up purposes.

    Args:
        data (np.ndarray): Array of shape (n, d). Training data set.
        classifications (np.ndarray): Array of shape (n,) full of integers in range
num_centers - 1}.
            Data point at index i is assigned to classifications[i].
        num_centers (int): Number of centers for reference.
            Might be usefull for pre-allocating numpy array (Faster that appending

    Returns:
        np.ndarray: Array of shape (num_centers, d) containing new centers.
    """
    return np.array([data[classifications == i].mean(axis=0) for i in range(num_cen
    new_centers = np.zeros((num_centers, data.shape[1]))
```

[1] To be more precise, it is both NP-hard in d when $k = 2$ and k when $d = 2$. See the references on the Wikipedia page for k-means for more details.

```
    for i in range(num_centers):
        idx = np.where(classifications == i)
        class_i = data[idx]
        new_center = np.mean(class_i, axis=0)
        new_centers[i] = new_center

    return new_centers


@problem.tag("hw4-A")
def cluster_data(data: np.ndarray, centers: np.ndarray) -> np.ndarray:
    """
    Sub-routine of Lloyd's algorithm that clusters datapoints to centers given data

    Args:
        data (np.ndarray): Array of shape (n, d). Training data set.
        centers (np.ndarray): Array of shape (k, d). Each row is a center to which

    Returns:
        np.ndarray: Array of integers of shape (n,), with each entry being in range
            Entry j at index i should mean that j^th center is the closest to data[
    """
    Centers = np.zeros(data.shape[0], dtype=np.int)
    for i in range(data.shape[0]):
        distances = np.linalg.norm(data[i] - centers, axis=1)
        center = np.argmin(distances)
        Centers[i] = center

    return Centers


@problem.tag("hw4-A")
def calculate_error(data: np.ndarray, centers: np.ndarray) -> float:
    """Calculates error/objective function on a provided dataset, with trained cent

    Args:
        data (np.ndarray): Array of shape (n, d). Dataset to evaluate centers on.
        centers (np.ndarray): Array of shape (k, d). Each row is a center to which
            These should be trained on training dataset.

    Returns:
        float: Single value representing mean objective function of centers on a pr
    """
    distances = np.zeros((data.shape[0], centers.shape[0]))
    for idx, center in enumerate(centers):
        distances[:, idx] = np.sqrt(np.sum((data - center) ** 2, axis=1))
    return np.mean(np.min(distances, axis=1))


@problem.tag("hw4-A")
def lloyd_algorithm(
    data: np.ndarray, num_centers: int, epsilon: float = 10e-3
) -> Tuple[np.ndarray, List[float]]:
    """Main part of Lloyd's Algorithm.
```

```
    Args:
        data (np.ndarray): Array of shape (n, d). Training data set.
        num_centers (int): Number of centers to train/cluster around.
        epsilon (float, optional): Epsilon for stopping condition.
            Training should stop when max(abs(centers - previous_centers)) is small
            Defaults to 10e-3.

    Returns:
        np.ndarray: Tuple of 2 numpy arrays:
            Element at index 0: Array of shape (num_centers, d) containing trained
            Element at index 1: List of floats of length # of iterations
                containing errors at the end of each iteration of lloyd's algorithm
                You should use the calculate_error() function that has been impleme

    Note:
        - For initializing centers please use the first `num_centers` data points.
    """
    centers = data[:num_centers]
    converged = False
    err = []

    while not converged:
        Class = cluster_data(data, centers)
        prev_centers = centers
        centers = calculate_centers(data, Class, num_centers)

        err.append(calculate_error(data, centers))

        if np.max(abs(centers - prev_centers)) < epsilon:
            converged = True

    return (centers, err)
```

b. *[5 points]* Run Lloyd's algorithm on the *training* dataset of MNIST with $k = 10$. Show the image representing the center of each cluster, as a set of k 28×28 images.

Note on Time to Run — The runtime of a good implementation for this problem should be fairly fast (a few minutes); if you find it taking upwards of one hour, please check your implementation! (Hint: **For loops are costly.** Can you vectorize it or use Numpy operations to make it faster in some ways? If not, is looping through data-points or through centers faster?)

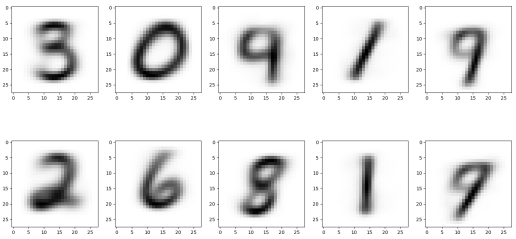


Figure 1: Cluster centers from Lloyd's algorithm with $k = 10$.

```
@problem.tag("hw4-A", start_line=1)
```

```
def main():
    """Main function of k-means problem

    Run Lloyd's Algorithm for k=10, and report 10 centers returned.

    NOTE: This code takes a while to run. For debugging purposes you might want to
        x_train to x_train[:10000]. CHANGE IT BACK before submission.
    """
    (x_train, _), (x_test, _) = load_dataset("mnist")
    centers, errors = lloyd_algorithm(x_train, 10)
    for i in range(10):
        image = centers[i].reshape((28, 28))
        plt.subplot(2, 5, i + 1)
        plt.imshow(image, cmap='Greys')

    plt.show()
```

Matrix Completion and Recommendation System

A4. You will build a personalized movie recommender system. We will use the 100K MovieLens dataset available at https://grouplens.org/datasets/movielens/100k/. There are $m = 1682$ movies and $n = 943$ users. Each user rated at least 20 movies, but some watched many more. The total dataset contains $100,000$ total ratings from all users. The goal is to recommend movies the users haven't seen.

Consider a matrix $R \in \mathbb{R}^{m \times n}$ where the entry $R_{i,j} \in \{1, \ldots, 5\}$ represents the jth user's rating on movie i. A higher value represents that the user is more satisfied with the movie.

We may think of our historical data as some observed entries of this matrix while many remain unknown, and we wish to estimate the unknown ratings that each user would assign to each movie. We could use these ratings to recommend the "best" movies for each user.

The dataset contains user and movie metadata which we will ignore. We strictly use the ratings contained in the `u.data` file. Use this data file and the following python code to construct a training and test set:

```
import csv
import numpy as np
data = []
with open('u.data') as csvfile:
    spamreader = csv.reader(csvfile, delimiter='\t')
    for row in spamreader:
        data.append([int(row[0])-1, int(row[1])-1, int(row[2])])
data = np.array(data)

num_observations = len(data)   # num_observations = 100,000
num_users = max(data[:,0])+1   # num_users = 943, indexed 0,...,942
num_items = max(data[:,1])+1   # num_items = 1682 indexed 0,...,1681

np.random.seed(1)
num_train = int(0.8*num_observations)
perm = np.random.permutation(data.shape[0])
train = data[perm[0:num_train],:]
test = data[perm[num_train::],:]
```

The arrays `train` and `test` contain R_{train} and R_{test}, respectively. Each line takes the form "j, i, s", where j is the user index, i is the movie index, and s is the user's score.

Using `train`, you will train a model that can predict $\widehat{R} \in \mathbb{R}^{m \times n}$, how every user would rate every movie. You will evaluate your model based on the average squared error on `test`:

$$\mathcal{E}_{\text{test}}(\widehat{R}) = \frac{1}{|\text{test}|} \sum_{(i,j,R_{i,j}) \in \text{test}} (\widehat{R}_{i,j} - R_{i,j})^2.$$

Low-rank matrix factorization is a baseline method for personalized recommendation. It learns a vector representation $u_i \in \mathbb{R}^d$ for each movie and a vector representation $v_j \in \mathbb{R}^d$ for each user, such that the inner product $\langle u_i, v_j \rangle$ approximates the rating $R_{i,j}$. You will build a simple latent factor model.

You will implement multiple estimators and use the inner product $\langle u_i, v_j \rangle$ to predict if user j likes movie i in the test data. For simplicity, we will put aside best practices and choose hyperparameters by using those that minimize the test error. You may use fundamental operators from `numpy` or `pytorch` in this problem (`numpy.linalg.lstsq, SVD, autograd,` etc.) but not any precooked algorithm from a package like `scikit-learn`. If there is a question whether some package is not allowed for use in this problem, it probably is not appropriate.

a. *[5 points]* Our first estimator pools all users together and, for each movie, outputs as its prediction the average user rating of that movie in `train`. That is, if $\mu \in \mathbb{R}^m$ is a vector where μ_i is the average rating of the users that rated the ith movie, write this estimator $\widehat{R}$ as a rank-one matrix.
Compute the estimate $\widehat{R}$. What is $\mathcal{E}_{\text{test}}(\widehat{R})$ for this estimate?

Using the average user ratings to predict new entries causes the error to simply be the variance of the matrix. The average rating of movies is computed by the following ratio:

$$\frac{\text{Total rating of given a movie}}{\text{Number of users that rated that movie}}$$

If that movie is not rated by any users, we can replace the average estimation to be the average of all rated movies. The final computed error was $\mathcal{E}_{\text{test}}(\widehat{R}) = 1.0636$.

```
# Compute estimate
print(np.max(train[:, 1]))

# Calculate average rating for each movie
movie_ratings_sum = np.zeros(num_items)
movie_ratings_count = np.zeros(num_items)

for user, movie, rating in train:
    movie_ratings_sum[movie] += rating
    movie_ratings_count[movie] += 1

movie_ratings_count[movie_ratings_count == 0] = 1
average_movie_ratings = movie_ratings_sum / movie_ratings_count

# Construct rank-one matrix R_hat
R_hat = np.tile(average_movie_ratings, (num_users, 1))

# Evaluate test error
test_error = 0
for user, movie, actual_rating in test:
    predicted_rating = R_hat[user, movie]
    test_error += (predicted_rating - actual_rating) ** 2

test_error /= len(test)
```

```
print("Test Error (MSE) for the estimator R_hat:", test_error)
```

b. *[5 points]* Allocate a matrix $\widetilde{R}_{i,j} \in \mathbb{R}^{m \times n}$ and set its entries equal to the known values in the training set, and 0 otherwise. Let $\widehat{R}^{(d)}$ be the best rank-d approximation (in terms of squared error) approximation to $\widetilde{R}$. This is equivalent to computing the singular value decomposition (SVD) and using the top d singular values. This learns a lower-dimensional vector representation for users and movies, assuming that each user would give a rating of 0 to any movie they have not reviewed.

- For each $d = 1, 2, 5, 10, 20, 50$, compute the estimator $\widehat{R}^{(d)}$. We recommend using an efficient solver such as `scipy.sparse.linalg.svds`.
- Plot the average squared error of predictions on the training set and test set on a single plot, as a function of d.

Note that, in most applications, we would not actually allocate a full $m \times n$ matrix. We do so here only because our data is relatively small and it is instructive.



Figure 2: Error of Rank(d) predictor vs. d.

```
# Create the matrix R twiddle (\widetilde{R})
R_tilde = np.zeros((num_users, num_items))
for user, movie, rating in train:
    R_tilde[user, movie] = rating

def construct_estimator(d, r_twiddle):
    U, s, Vt = svds(r_twiddle, k = d)
    R_hat_d = U @ np.diag(s) @ Vt

    return R_hat_d


def get_error(d, r_twiddle, dataset):
    R_hat_d = construct_estimator(d, r_twiddle)
    error = 0
    for user, movie, actual_rating in dataset:
```

```
            predicted_rating = R_hat_d[user, movie]
            error += (predicted_rating - actual_rating) ** 2
    return error / len(dataset)

# Evaluate train and test error for: d = 1, 2, 5, 10, 20, 50
d_values = [1, 2, 5, 10, 20, 50]
usv_train_errors = []
usv_test_errors = []
for d in d_values:
    usv_train_error=get_error(d, R_tilde, train)
    usv_test_error=get_error(d, R_tilde, test)
    usv_train_errors.append(usv_train_error)
    usv_test_errors.append(usv_test_error)

# Plot both train and test error as a function of d on the same plot
plt.figure(figsize=(10, 6))
plt.plot(d_values, usv_train_errors, label='Training Error', marker='o')
plt.plot(d_values, usv_test_errors, label='Test Error', marker='o')
plt.xlabel('d (Number of Singular Values)')
plt.ylabel('Average Squared Error')
plt.title('Training and Test Errors vs. Rank of Approximation')
plt.legend()
plt.grid(True)
plt.show()
```

c. *[10 points]* Replacing all missing values by a constant may impose strong and potentially incorrect assumptions on the unobserved entries of R. A more reasonable choice is to minimize the MSE (mean squared error) only on rated movies. Define a loss function:

$$\mathcal{L}\Big(\{u_i\}_{i=1}^m, \{v_j\}_{j=1}^n\Big) := \sum_{(i,j,R_{i,j})\in \text{train}} (\langle u_i, v_j\rangle - R_{i,j})^2 + \lambda \sum_{i=1}^m \|u_i\|_2^2 + \lambda \sum_{j=1}^n \|v_j\|_2^2 \tag{2}$$

where $\lambda > 0$ is the regularization coefficient. We will implement algorithms to learn vector representations by minimizing (2).

Since this is a non-convex optimization problem, the initial starting point and hyperparameters may affect the quality of $\widehat{R}$. You may need to tune λ and σ to optimize the loss you see.

- *Alternating minimization*: First, randomly initialize $\{u_i\}$ and $\{v_j\}$. Then, alternatate between (1) minimizing the loss function with respect to $\{u_i\}$ by treating $\{v_j\}$ as fixed; and (2) minimizing the loss function with respect to $\{v_j\}$ by treating $\{u_i\}$ as fixed. Repeat (1) and (2) until both $\{u_i\}$ and $\{v_j\}$ converge. Note that when one of $\{u_i\}$ or $\{v_j\}$ is given, minimizing the loss function with respect to the other part has a closed-form solution. Indeed, it can be shown that when minimizing with respect to a *single* u_i (with $\{v_j\}$ fixed), the gradient is given by:

$$\nabla_{u_i} L\Big(\{u_i\}_{i=1}^m, \{v_j\}_{j=1}^n\Big) = 2\left(\sum_{j\in r(i)} v_j v_j^T + \lambda I\right) u_i - 2\sum_{j\in r(i)} R_{i,j} v_j \tag{3}$$

where here $r(i)$ is a shorthand for the set of users who have reviewed movie i in the training set, or more formally, $r(i) = \{j : (j, i, R_{i,j}) \in \text{train}\}$. Setting the overall gradient to be equal to 0 gives us that

$$\arg\min_{u_i} L\Big(\{u_i\}_{i=1}^m, \{v_j\}_{j=1}^n\Big) = \left(\sum_{j\in r(i)} v_j v_j^T + \lambda I\right)^{-1} \left(\sum_{j\in r(i)} R_{i,j} v_j\right) \tag{4}$$

Note that this update rule is for a single vector u_i, whereas you should update all of the $\{u_i\}_{i=1}^m$ in one round. When it comes to the alternate step which involves fixing $\{u_i\}$ and minimizing $\{v_j\}$, an analogous calculation will give you a very similar update rule.

- Try $d \in \{1, 2, 5, 10, 20, 50\}$ and plot the mean squared error of train and test as a function of d.

Some hints:

- Common choices for initializing the vectors $\{u_i\}_{i=1}^m$, $\{v_j\}_{j=1}^n$ include: entries drawn from `np.random.rand()` scaled by some scale factor $\sigma > 0$ (σ is an additional hyperparameter), or using one of the solutions from part b or c.
- You should never be allocating an $m \times n$ matrix for this problem.
- It is **crucial** that the squared error part of the loss is only defined w.r.t. $R_{i,j}$ that actually exist in the training set. Consider implementing some type of data structures that allow you to keep track of $r(i)$ as well as the reverse mapping $r^{-1}(j)$ from movies to relevant users.

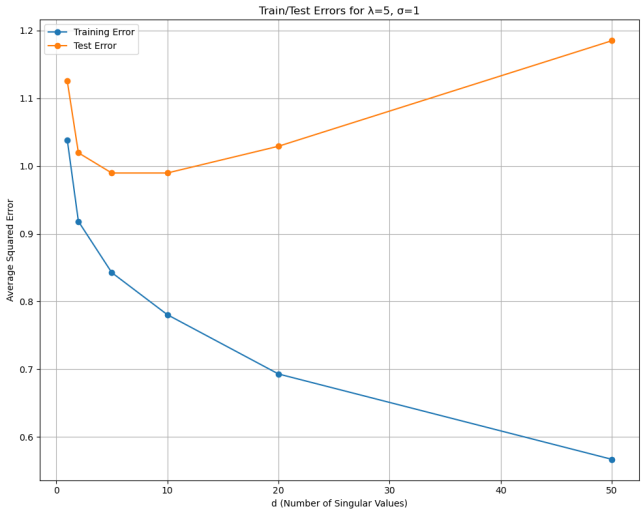


Figure 3: Error of Rank(d) predictor vs. d with regularization for $\lambda = 5$ and $\sigma = 1$.

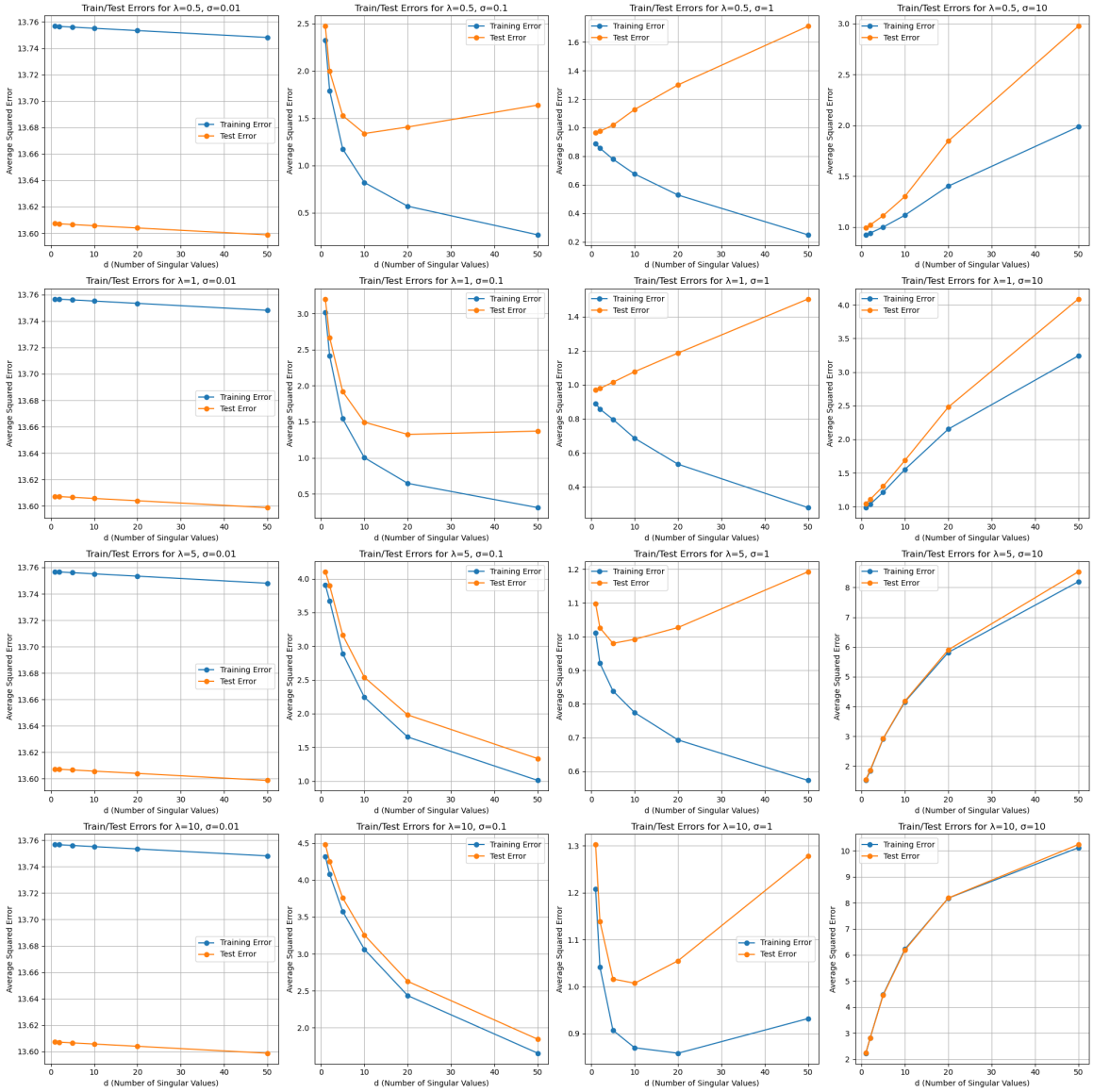


Figure 4: Error of Rank(d) predictor vs. d with regularization for combinations of λ and σ.

```
def closed_form_u(V, U, l, R_tilde):
    print(V.shape, U.shape)
    for i in range(U.shape[0]):
        R_i_tile = R_tilde[i]
        indices = R_i_tile > 0
        V_j = V[indices, :]
        R_i = R_i_tile[indices]
        A = V_j.T @ V_j + l * np.eye(V_j.shape[1])
        b = (V_j.T @ R_i).reshape(-1, 1)
        U[i,:] = np.linalg.solve(A, b).flatten()
    return U


def closed_form_v(V, U, l, R_tilde):
    for j in range(V.shape[0]):
```

```
        R_j_tile = R_tilde[:,j]
        indices = R_j_tile > 0
        U_i = U[indices, :]
        R_j = R_j_tile[indices]
        A = U_i.T @ U_i + l * np.eye(U_i.shape[1])
        b = (U_i.T @ R_j).reshape(-1, 1)
        V[j,:] = np.linalg.solve(A, b).flatten()
    return V


def construct_alternating_estimator(d, r_twiddle, l=0.0, delta=1e-2, sigma=0.1, U=None,
    old_U, old_V = np.zeros((num_users, d)), np.zeros((num_items, d))
    if U is None:
        U = np.random.rand(num_users, d) * sigma
    if V is None:
        V = np.random.rand(num_items, d) * sigma
    while (np.max(np.abs(V - old_V)) > delta and np.max(np.abs(U - old_U)) > delta):
        old_U, old_V = U, V
        U = closed_form_u(V, U, l, r_twiddle)
        V = closed_form_v(V, U, l, r_twiddle)
    return U, V

def calc_uv_error(dataset, U, V):
    user = dataset[:,0]
    item = dataset[:,1]
    score = dataset[:,2]
    pred = np.einsum('ij,ij->i', U[user], V[item])
    mse_error = np.mean((score-pred) ** 2)
    return mse_error

# Evaluate train and test error for: d = 1, 2, 5, 10, 20, 50
from itertools import product
d_vals = [1, 2, 5, 10, 20, 50]
lambdas = [0.5, 1, 5, 10]
sigmas = [0.01, 0.1, 1, 10]

# Plot both train and test error as a function of d on the same plot
fig, axes = plt.subplots(4, 4, figsize=(20, 20))
axes = axes.flatten()

# Iterate over all combinations of lambdas and sigmas
for index, (lambda_val, sigma_val) in enumerate(product(lambdas, sigmas)):
    uv_train_errors = []
    uv_test_errors = []

    for d in d_vals:
        U, V = construct_alternating_estimator(d=d, r_twiddle=R_tilde, l=lambda_val, sig
        uv_train_errors.append(calc_uv_error(train, U, V))
        uv_test_errors.append(calc_uv_error(test, U, V))

    # Plot the train and test error for each combination
    ax = axes[index]
    ax.plot(d_vals, uv_train_errors, label='Training Error', marker='o')
    ax.plot(d_vals, uv_test_errors, label='Test Error', marker='o')
    ax.set_xlabel('d (Number of Singular Values)')
```

```
        ax.set_ylabel('Average Squared Error')
        ax.set_title(f'Train/Test Errors for  ={lambda_val},  ={sigma_val}')
        ax.legend()
        ax.grid(True)

    plt.tight_layout()
    plt.show()
```

Image Classification on CIFAR-10

A5. In this problem we will explore different deep learning architectures for image classification on the CIFAR-10 dataset. Make sure that you are familiar with `torch.Tensors`, two-dimensional convolutions (`nn.Conv2d`) and fully-connected layers (`nn.Linear`), ReLU non-linearities (`F.relu`), pooling (`nn.MaxPool2d`), and tensor reshaping (`view`).

A few preliminaries:

- Each network f maps an image $x^{\text{in}} \in \mathbb{R}^{32\times32\times3}$ (3 channels for RGB) to an output $f(x^{\text{in}}) = x^{\text{out}} \in \mathbb{R}^{10}$. The class label is predicted as $\arg\max_{i=0,1,\ldots,9} x_i^{\text{out}}$. An error occurs if the predicted label differs from the true label for a given image.
- The network is trained via multiclass cross-entropy loss.
- Create a validation dataset by appropriately partitioning the train dataset. *Hint*: look at the documentation for `torch.utils.data.random_split`. Make sure to tune hyperparameters like network architecture and step size on the validation dataset. Do **NOT** validate your hyperparameters on the test dataset.
- At the end of each epoch (one pass over the training data), compute and print the training and validation classification accuracy.
- While one would usually train a network for hundreds of epochs to reach convergence and maximize accuracy, this can be prohibitively time-consuming, so feel free to train for just a dozen or so epochs.

For parts (a) and (b), apply a hyperparameter tuning method (e.g. random search, grid search, etc.) using the validation set, report the hyperparameter configurations you evaluated and the best set of hyperparameters from this set, and plot the training and validation classification accuracy as a function of epochs. Produce a separate line or plot for each hyperparameter configuration evaluated (top 3 configurations is sufficient to keep the plots clean). Finally, evaluate your best set of hyperparameters on the test data and report the test accuracy.

Note 1: Please refer to the provided notebook with starter code for this problem, included with the code files for this assignment. That notebook provides a complete end-to-end example of loading data, training a model using a simple network with a fully-connected output and no hidden layers (this is equivalent to logistic regression), and performing evaluation using canonical Pytorch. We recommend using this as a template for your implementations of the models below.
Note 2: If you are attempting this problem and do not have access to GPU we highly recommend using Google Colab. The provided notebook includes instructions on how to use GPU in Google Colab.

Here are the network architectures you will construct and compare.

a. *[18 points]* **Fully-connected output, 1 fully-connected hidden layer:** this network has one hidden layer denoted as $x^{\text{hidden}} \in \mathbb{R}^M$ where M will be a hyperparameter you choose (M could be in the hundreds). The nonlinearity applied to the hidden layer will be the `relu` ($\text{relu}(x) = \max\{0, x\}$. This network can be written as

$$x^{out} = W_2\text{relu}(W_1(x^{in}) + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{M \times 3072}$, $b_1 \in \mathbb{R}^M$, $W_2 \in \mathbb{R}^{10 \times M}$, $b_2 \in \mathbb{R}^{10}$. Tune the different hyperparameters and train for a sufficient number of epochs to achieve a *validation accuracy* of at least 50%. Provide the hyperparameter configuration used to achieve this performance.

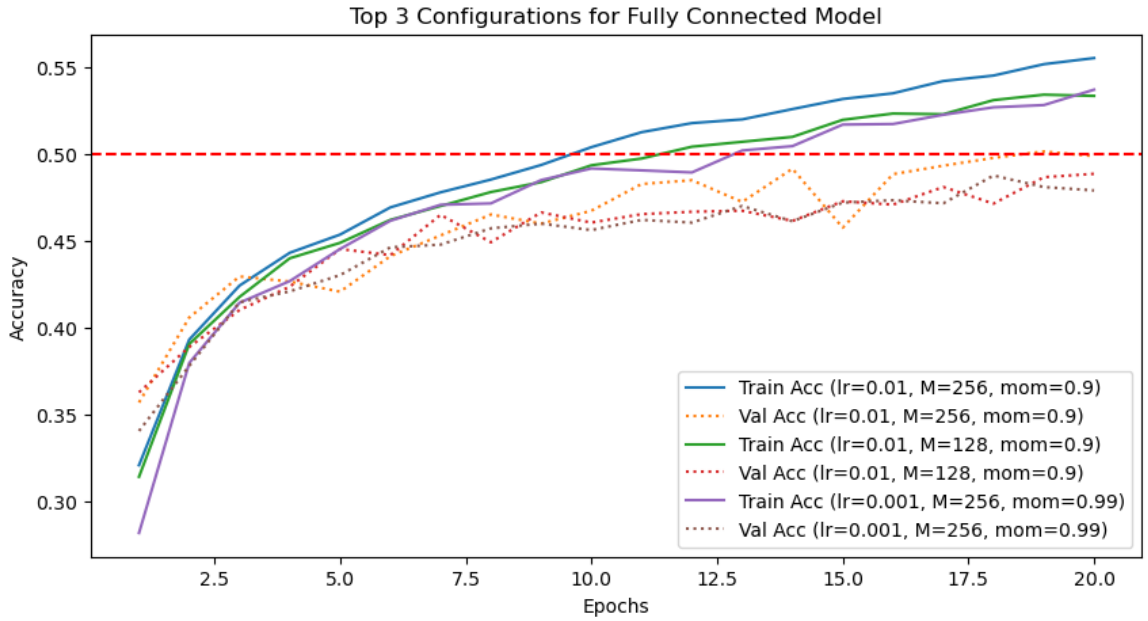


Figure 5: Training and validation accuracy for the top three hyperparameter configurations for the fully-connected neural network model.

I used grid search with the following hyperparameter values:

Hyperparameter	**Values Tested**
Learning Rate	0.001, 0.01
Hidden Layer Dimension	64, 128, 256
Momentum	0.9, 0.99
Epochs	20

The best-performing hyperparameter values, and the resulting test accuracy, were as follows:

Learning Rate	Hidden Layer Dimension	Momentum	Test Accuracy
0.01	256	0.9	0.504746835443038
0.01	128	0.9	0.5014833860759493
0.001	256	0.99	0.4957476265822785

b. *[18 points]* **Convolutional layer with max-pool and fully-connected output:** for a convolutional layer W_1 with filters of size $k \times k \times 3$, and M filters (reasonable choices are $M = 100$, $k = 5$), we have that $\text{Conv2d}(x^{\text{in}}, W_1) \in \mathbb{R}^{(33-k)\times(33-k)\times M}$.

- Each convolution will have its own offset applied to each of the output pixels of the convolution; we denote this as $\text{Conv2d}(x^{\text{in}}, W) + b_1$ where b_1 is parameterized in $\mathbb{R}^M$. Apply a `relu` activation to the result of the convolutional layer.
- Next, use a max-pool of size $N \times N$ (a reasonable choice is $N = 14$ to pool to 2×2 with $k = 5$) we have that $\text{MaxPool}(\text{relu}(\text{Conv2d}(x^{\text{in}}, W_1) + b_1)) \in \mathbb{R}^{\lfloor \frac{33-k}{N} \rfloor \times \lfloor \frac{33-k}{N} \rfloor \times M}$.
- We will then apply a fully-connected layer to the output to get a final network given as

$$x^{output} = W_2(\text{MaxPool}(\text{relu}(\text{Conv2d}(x^{\text{input}}, W_1) + b_1))) + b_2$$

where $W_2 \in \mathbb{R}^{10 \times M(\lfloor \frac{33-k}{N} \rfloor)^2}$, $b_2 \in \mathbb{R}^{10}$.

The parameters M, k, N (in addition to the step size and momentum) are all hyperparameters, but you can choose a reasonable value. Tune the different hyperparameters (number of convolutional filters, filter sizes, dimensionality of the fully-connected layers, step size, etc.) and train for a sufficient number of epochs to achieve a *validation accuracy* of at least 65%. Provide the hyperparameter configuration used to achieve this performance.

The number of hyperparameters to tune, combined with the slow training times, will hopefully give you a taste of how difficult it is to construct networks with good generalization performance. State-of-the-art networks can have dozens of layers, each with their own hyperparameters to tune. Additional hyperparameters you are welcome to play with if you are so inclined, include: changing the activation function, replace max-pool with average-pool, adding more convolutional or fully connected layers, and experimenting with batch normalization or dropout.

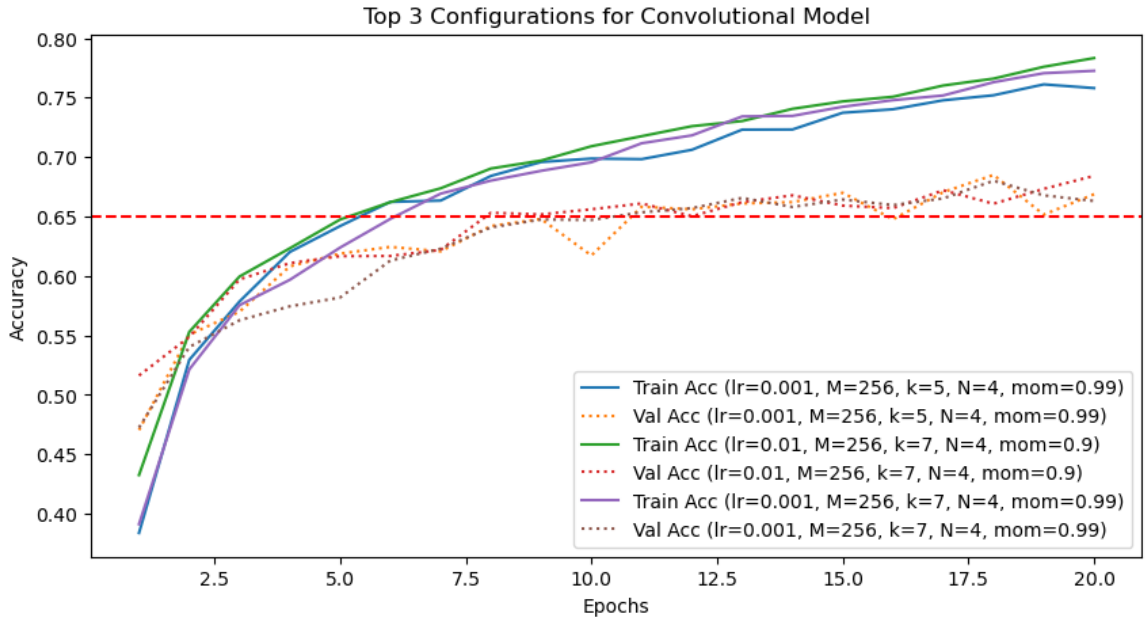


Figure 6: Training and validation accuracy for the top three hyperparameter configurations for the convolutional neural network model.

I used grid search with the following hyperparameter values:

Hyperparameter	**Values Tested**
Learning Rate	0.001, 0.01
Hidden Layer Dimension	64, 128, 256
Kernal Size	3, 5, 7
Pool Size	2, 4
Momentum	0.9, 0.99
Epochs	20

The best-performing hyperparameter values, and the resulting test accuracy, were as follows:

Learning Rate	Hidden Layer Dimension	Kernal Size	Pool Size	Momentum	Test Accuracy
0.001	256	5	4	0.99	0.6811708860759493
0.01	256	7	4	0.9	0.6830498417721519
0.001	256	7	4	0.99	0.6771162974683544

```
import torch
from torch import nn
import numpy as np

from typing import Tuple, Union, List, Callable
from torch.optim import SGD
import torchvision
from torch.utils.data import DataLoader, TensorDataset, random_split
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm

assert torch.cuda.is_available(), "GPU is not available, check the directions above (or disa

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(DEVICE)  # this should print out CUDA

train_dataset = torchvision.datasets.CIFAR10("./data", train=True, download=True, transform=
test_dataset = torchvision.datasets.CIFAR10("./data", train=False, download=True, transform=

batch_size = 128

train_dataset, val_dataset = random_split(train_dataset, [int(0.9 * len(train_dataset)), int

# Create separate dataloaders for the train, test, and validation set
train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=8,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=8,
    pin_memory=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=8,
    pin_memory=True
)

imgs, labels = next(iter(train_loader))
print(f"A single batch of images has shape: {imgs.size()}")
example_image, example_label = imgs[0], labels[0]
c, w, h = example_image.size()
print(f"A single RGB image has {c} channels, width {w}, and height {h}.")

# This is one way to flatten our images
```

```
batch_flat_view = imgs.view(-1, c * w * h)
print(f"Size of a batch of images flattened with view: {batch_flat_view.size()}")

# This is another equivalent way
batch_flat_flatten = imgs.flatten(1)
print(f"Size of a batch of images flattened with flatten: {batch_flat_flatten.size()}")

# The new dimension is just the product of the ones we flattened
d = example_image.flatten().size()[0]
print(c * w * h == d)

# View the image
t =  torchvision.transforms.ToPILImage()
plt.imshow(t(example_image))

# These are what the class labels in CIFAR-10 represent. For more information,
# visit https://www.cs.toronto.edu/~kriz/cifar.html
classes = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog",
           "horse", "ship", "truck"]
print(f"This image is labeled as class {classes[example_label]}")

def linear_model() -> nn.Module:
    """Instantiate a linear model and send it to device."""
    model =  nn.Sequential(
            nn.Flatten(),
            nn.Linear(d, 10)
        )
    return model.to(DEVICE)

def train(
    model: nn.Module, optimizer: SGD,
    train_loader: DataLoader, val_loader: DataLoader,
    epochs: int = 20
)-> Tuple[List[float], List[float], List[float], List[float]]:
    """
    Trains a model for the specified number of epochs using the loaders.

    Returns:
    Lists of training loss, training accuracy, validation loss, validation accuracy for each
    """

    loss = nn.CrossEntropyLoss()
    train_losses = []
    train_accuracies = []
    val_losses = []
    val_accuracies = []
    for e in tqdm(range(epochs)):
        model.train()
        train_loss = 0.0
        train_acc = 0.0

        # Main training loop; iterate over train_loader. The loop
        # terminates when the train loader finishes iterating, which is one epoch.
        for (x_batch, labels) in train_loader:
            x_batch, labels = x_batch.to(DEVICE), labels.to(DEVICE)
```

```
            optimizer.zero_grad()
            labels_pred = model(x_batch)
            batch_loss = loss(labels_pred, labels)
            train_loss = train_loss + batch_loss.item()

            labels_pred_max = torch.argmax(labels_pred, 1)
            batch_acc = torch.sum(labels_pred_max == labels)
            train_acc = train_acc + batch_acc.item()

            batch_loss.backward()
            optimizer.step()
        train_losses.append(train_loss / len(train_loader))
        train_accuracies.append(train_acc / (batch_size * len(train_loader)))

        # Validation loop; use .no_grad() context manager to save memory.
        model.eval()
        val_loss = 0.0
        val_acc = 0.0

        with torch.no_grad():
            for (v_batch, labels) in val_loader:
                v_batch, labels = v_batch.to(DEVICE), labels.to(DEVICE)
                labels_pred = model(v_batch)
                v_batch_loss = loss(labels_pred, labels)
                val_loss = val_loss + v_batch_loss.item()

                v_pred_max = torch.argmax(labels_pred, 1)
                batch_acc = torch.sum(v_pred_max == labels)
                val_acc = val_acc + batch_acc.item()
            val_losses.append(val_loss / len(val_loader))
            val_accuracies.append(val_acc / (batch_size * len(val_loader)))

    return train_losses, train_accuracies, val_losses, val_accuracies

def parameter_search(
    train_loader: DataLoader,
    val_loader: DataLoader,
    model_fn:Callable[[], nn.Module]
) -> float:
    """
    Parameter search for our linear model using SGD.

    Args:
    train_loader: the train dataloader.
    val_loader: the validation dataloader.
    model_fn: a function that, when called, returns a torch.nn.Module.

    Returns:
    The learning rate with the least validation loss.
    NOTE: you may need to modify this function to search over and return
     other parameters beyond learning rate.
    """
    num_iter = 10
    best_loss = torch.tensor(np.inf)
    best_lr = 0.0
```

```
    lrs = torch.linspace(10 ** (-6), 10 ** (-1), num_iter)

    for lr in lrs:
        print(f"trying learning rate {lr}")
        model = model_fn()
        optim = SGD(model.parameters(), lr)

        train_loss, train_acc, val_loss, val_acc = train(
            model,
            optim,
            train_loader,
            val_loader,
            epochs=20
            )

        if min(val_loss) < best_loss:
            best_loss = min(val_loss)
            best_lr = lr

    return best_lr

best_lr = parameter_search(train_loader, val_loader, linear_model)

model = linear_model()
optimizer = SGD(model.parameters(), best_lr)

# We are only using 20 epochs for this example. You may have to use more.
train_loss, train_accuracy, val_loss, val_accuracy = train(
    model, optimizer, train_loader, val_loader, 20
)

epochs = range(1, 21)
plt.plot(epochs, train_accuracy, label="Train Accuracy")
plt.plot(epochs, val_accuracy, label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Logistic Regression Accuracy for CIFAR-10 vs Epoch")
plt.show()

def evaluate(
    model: nn.Module, loader: DataLoader
) -> Tuple[float, float]:
    """Computes test loss and accuracy of model on loader."""
    loss = nn.CrossEntropyLoss()
    model.eval()
    test_loss = 0.0
    test_acc = 0.0
    with torch.no_grad():
        for (batch, labels) in loader:
            batch, labels = batch.to(DEVICE), labels.to(DEVICE)
            y_batch_pred = model(batch)
            batch_loss = loss(y_batch_pred, labels)
            test_loss = test_loss + batch_loss.item()
```

```
            pred_max = torch.argmax(y_batch_pred, 1)
            batch_acc = torch.sum(pred_max == labels)
            test_acc = test_acc + batch_acc.item()
        test_loss = test_loss / len(loader)
        test_acc = test_acc / (batch_size * len(loader))
        return test_loss, test_acc

test_loss, test_acc = evaluate(model, test_loader)
print(f"Test Accuracy: {test_acc}")

def fully_connected_model(M, input_dim=3072, output_dim=10):
    """Instantiate a fully connected neural network model and send it to device."""
    model = nn.Sequential(
        nn.Flatten(),
        nn.Linear(input_dim, M),
        nn.ReLU(),
        nn.Linear(M, output_dim)
    )
    return model.to(DEVICE)

def conv_model(M, k, N, input_size=(3,32,32), output_dim=10):
    """Instantiate a convolutional neural network model and send it to device."""
    model = nn.Sequential(
        nn.Conv2d(input_size[0], M, k, padding=k//2),
        nn.ReLU(),
        nn.MaxPool2d(N),
        nn.Flatten(),
        nn.Linear(M * ((input_size[1] // N) ** 2), output_dim)
    )
    return model.to(DEVICE)

def parameter_search(
    train_loader: DataLoader,
    val_loader: DataLoader,
    model_fn: Callable[[], nn.Module]
) -> List[Tuple[float, float, float, int, int, int, float]]:
    """
    Parameter search for our fully connected and convolutional neural network models using S

    Args:
    train_loader: the train dataloader.
    val_loader: the validation dataloader.
    model_fn: a function that, when called, returns a torch.nn.Module.

    Returns:
    The top three hyperparameter configurations.
    """
    results = []
    num_epochs = 20

    lrs = [0.001, 0.01]
    Ms = [64, 128, 256]
    momentums = [0.9, 0.99]
```

```
    if "fully_connected" in model_fn.__name__:
        for lr in lrs:
            for M in Ms:
                for momentum in momentums:
                    model = model_fn(M)
                    optimizer = SGD(model.parameters(), lr=lr, momentum=momentum)
                    train_loss, train_acc, val_loss, val_acc = train(model, optimizer, train
                    print(f'Fully Connected Model - Result for learning rate: {lr}, hidden l
                    results.append((train_acc, val_acc, lr, M, momentum))

    elif "conv" in model_fn.__name__:
        ks = [3, 5, 7]
        Ns = [2, 4]

        for lr in lrs:
            for M in Ms:
                for k in ks:
                    for N in Ns:
                        for momentum in momentums:
                            model = model_fn(M, k, N)
                            optimizer = SGD(model.parameters(), lr=lr, momentum=momentum)
                            train_loss, train_acc, val_loss, val_acc = train(model, optimize
                            print(f'Convolutional Model - Result for learning rate: {lr}, hi
                            results.append((train_acc, val_acc, lr, M, k, N, momentum))

    results.sort(key=lambda x: max(x[1]), reverse=True)

    return results[:3]

# Retrieve top three hyperparameter configurations for each model
fully_connected_results = parameter_search(train_loader, val_loader, fully_connected_model)
conv_results = parameter_search(train_loader, val_loader, conv_model)

def plot_results(results, model_name, threshold):
    plt.figure(figsize=(10, 5))

    for result in results:
        if model_name == "Fully Connected Model":
            train_acc, val_acc, lr, M, momentum = result
            train_label = f"Train Acc (lr={lr}, M={M}, mom={momentum})"
            val_label = f"Val Acc (lr={lr}, M={M}, mom={momentum})"
        elif model_name == "Convolutional Model":
            train_acc, val_acc, lr, M, k, N, momentum = result
            train_label = f"Train Acc (lr={lr}, M={M}, k={k}, N={N}, mom={momentum})"
            val_label = f"Val Acc (lr={lr}, M={M}, k={k}, N={N}, mom={momentum})"

        epochs = range(1, len(train_acc) + 1)
        plt.plot(epochs, train_acc, label=train_label)
        plt.plot(epochs, val_acc, linestyle='dotted', label=val_label)

    plt.axhline(y=threshold, linestyle="--", color='red')
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.title(f"Top 3 Configurations for {model_name}")
    plt.legend()
```

```
    plt.show()

plot_results(fully_connected_results, "Fully Connected Model", threshold=0.50)
plot_results(conv_results, "Convolutional Model", threshold=0.65)

# Train and evaluate the top 3 configurations for the fully connected model
for i, (fc_train_acc, fc_val_acc, fc_lr, fc_M, fc_momentum) in enumerate(fully_connected_res
    fully_connected_model_instance = fully_connected_model(fc_M)
    fc_optimizer = SGD(fully_connected_model_instance.parameters(), lr=fc_lr, momentum=fc_mo
    train(fully_connected_model_instance, fc_optimizer, train_loader, val_loader, epochs=100
    fc_test_loss, fc_test_acc = evaluate(fully_connected_model_instance, test_loader)
    print(f"Fully Connected Model {i+1} - Learning Rate: {fc_lr}, Hidden Layer: {fc_M}, Mom

# Train and evaluate the top 3 configurations for the convolutional model
for i, (conv_train_acc, conv_val_acc, conv_lr, conv_M, conv_k, conv_N, conv_momentum) in enu
    conv_model_instance = conv_model(conv_M, conv_k, conv_N)
    conv_optimizer = SGD(conv_model_instance.parameters(), lr=conv_lr, momentum=conv_momentu
    train(conv_model_instance, conv_optimizer, train_loader, val_loader, epochs=100)
    conv_test_loss, conv_test_acc = evaluate(conv_model_instance, test_loader)
    print(f"Convolutional Model {i+1} - Learning Rate: {conv_lr}, Hidden Layer: {conv_M}, K
```

Random Fourier Features

B1. Kernel methods such as Support Vector Machines are considered memory-based learners. Rather than learning a mapping from a set of input features $\mathcal{X} \subset \mathbb{R}^d$ to outputs in $\mathcal{Y}$, they *remember* all training examples $(\mathbf{x_i}, y_i)$ and learn a corresponding weight for them.

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^{N} \omega_i k(\mathbf{x_i}, \mathbf{x})$$

After learning the weight vector $\mathbf{w} = [\mathbf{w}_1, ..., \mathbf{w}_N]$, we can make prediction on unseen samples using the *kernel function* k between all training samples and $\mathbf{x}$. Kernel methods are attractive because they rely on the *kernel trick*. Any positive definite function $k(\mathbf{x}, \mathbf{x}')$ with $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ defines a function ψ mapping $\mathbb{R}^d$ to a higher-dimensional space such that the inner product between datapoints can be quickly computed as $\langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle = k(\mathbf{x}, \mathbf{x}')$. In essence, the kernel trick is an efficient way to learn a linear decision boundary in a higher dimension space than that of $\mathcal{X}$.

The kernel trick can be prohibitively expensive for large datasets. This is because the memory-based algorithm accesses the data through evaluations of the kernel matrix $k(x, x')$ which grows in proportion to the dataset size N.
Instead of relying on the implicit feature mapping ψ provided by the kernel trick, suppose we can approximate the kernel function k as the inner product of two vectors in $\mathbb{R}^D$. Mathematically, we would like to find a mapping $\mathbf{z}$:

$$\mathbf{z} : \mathbb{R}^d \to \mathbb{R}^D \qquad \text{such that} \qquad k_p(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle \approx \langle \mathbf{z}(\mathbf{x}), \mathbf{z}(\mathbf{x}') \rangle$$

With this approximation, we no longer require the *kernel trick* to express $\langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$ as $k(\mathbf{x}, \mathbf{x}')$. Rather, we can approximate it by directly computing the tractable inner product $\langle \mathbf{z}(\mathbf{x}), \mathbf{z}(\mathbf{x}') \rangle$.

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^{N} \omega_i k(\mathbf{x}_i, \mathbf{x}) = \sum_{i=1}^{N} \omega_i \langle \psi(\mathbf{x}_i), \psi(\mathbf{x}) \rangle \approx \sum_{i=1}^{N} \omega_i \langle \mathbf{z}(\mathbf{x}), \mathbf{z}(\mathbf{x}) \rangle = \left(\sum_{i=1}^{N} \omega_i \mathbf{z}(\mathbf{x}_i)^T \right) \mathbf{z}(\mathbf{x}) = \beta^T \mathbf{z}(\mathbf{x})$$

Assuming $\mathbf{z}(\mathbf{x}) = \sigma(M\mathbf{x} + b)$ for some nonlinear function σ, this "approximate" SVM *can potentially be evaluated much quicker* than the kernel SVM. To see why, note that the left-hand-side requires evaluating $k(\mathbf{x_i}, \mathbf{x})$ for all $i \in \{1, \ldots, N\}$, in general, if ω_i is not sparse. On the other hand, the right-hand-side just requires computing $\mathbf{z}(\mathbf{x}) = \sigma(M\mathbf{x} + b)$ which is dominated by the time to compute a $D \times d$ matrix-vector product, and then inner product with β which is $\mathbb{R}^D$. Thus, the total computation time for the left-hand-side scales linearly with N, and the right-hand-side scales with just d and D, independent of N! When training the approximate SVM we also get similar computational savings if $N \gg \max\{d, D\}$.

a. *[15 points]* **Deriving Random Fourier Features**: Bochner's theorem states that a continuous kernel $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x} - \mathbf{x}')$ on $\mathbb{R}^d$ is positive definite if and only if k is the Fourier transform of a non-negative measure. While we won't delve into the logic of Fourier transforms here, this theorem lets us express the kernel as follows: for any probability distribution $p(\mathbf{w})$ define

$$k_p(\mathbf{x}, \mathbf{x}') := \int_{\mathbb{R}^d} p(\mathbf{w}) e^{i\mathbf{w}^T(\mathbf{x}-\mathbf{x}')} dw = \mathbb{E}_{\mathbf{w}}\left[e^{i\mathbf{w}^T(\mathbf{x}-\mathbf{x}')}\right]$$

where $i = \sqrt{-1}$, the imaginary unit. While any choice of $p(\mathbf{w})$ induces a valid kernel, in this problem we'll be focusing on the Gaussian distribution, namely

$$p(\mathbf{w}) = (2\pi\sigma^2)^{-\frac{D}{2}} e^{-\frac{1}{2\sigma^2}||\mathbf{w}||_2^2} = (2\pi/\gamma^2)^{-\frac{D}{2}} e^{-\gamma^2||\mathbf{w}||_2^2/2} \quad \text{where } \gamma = \frac{1}{\sigma}$$

In this sub-problem, we'll use this Fourier-transform interpretation of k to derive a randomized mapping $\mathbf{z} : \mathbb{R}^d \to \mathbb{R}^D$ which is an unbiased estimate of the kernel function i.e.

$$\mathbb{E}_{\mathbf{w}}[\mathbf{z}(\mathbf{x})^T \mathbf{z}(\mathbf{x}')] = k_p(\mathbf{x}, \mathbf{x}')$$

If $\mathbf{z}(x)^T \mathbf{z}(x')$ serves as a good approximation to the kernel matrix, we can apply the aforementioned approximation algorithm.

i Use Euler's formula $e^{iy} = \cos(y) + i\sin(y)$ to show that $k_p(\mathbf{x}, \mathbf{x}') = E_{\mathbf{w}}\left[\cos(\mathbf{w}^T(\mathbf{x} - \mathbf{x}'))\right]$.

Hint: If both x and A are real, then $A = \int f(x) + ig(x)dx = \int f(x)dx$.

ii We begin by defining $z_{\mathbf{w}} : \mathbb{R}^d \to \mathbb{R}$ as

$$z_{\mathbf{w}}(\mathbf{x}) = \sqrt{2}\cos(\mathbf{w}^T\mathbf{x} + b) \qquad \text{where } \mathbf{w} \sim p(\mathbf{w}),\ b \sim \text{Uniform}(0, 2\pi)$$

Note that this is not yet the mapping vector $\mathbf{z}$, but rather a mapping to $\mathbb{R}$. Use part (i) to show that the expected product of $z_{\mathbf{w}}(\mathbf{x})$s is an unbiased estimate of the kernel function i.e.

$$E_{\mathbf{w},b}\left[z_{\mathbf{w}}(\mathbf{x}) z_{\mathbf{w}}(\mathbf{x}')\right] = k_p(\mathbf{x}, \mathbf{x}')$$

Hint: For this problem you may use the following identity: $2\cos(\alpha)\cos(\beta) = \cos(\alpha+\beta) + \cos(\alpha-\beta)$.

iii Now we're ready to define our random Fourier features $\mathbf{z} : \mathbb{R}^d \to \mathbb{R}^D$. Let $\mathbf{z}$ be the d-dimensional concatenation of $z_{\mathbf{w}}(\mathbf{x})$s:

$$\mathbf{z}(\mathbf{x}) = \left[\tfrac{1}{\sqrt{D}} z_{\mathbf{w}_1}(\mathbf{x}),\ \tfrac{1}{\sqrt{D}} z_{\mathbf{w}_2}(\mathbf{x}),\ \ldots,\ \tfrac{1}{\sqrt{D}} z_{\mathbf{w}_D}(\mathbf{x})\right]^T$$

Use parts (i) and (ii) to show that the expected inner product of the mapping $\mathbf{z}$ is an unbiased estimate of the kernel function i.e.

$$E_{\mathbf{w}}[\mathbf{z}(\mathbf{x})^T \mathbf{z}(\mathbf{x}')] = k_p(\mathbf{x}, \mathbf{x}')$$

i : Show that $k_p(x, x') = \mathbb{E}_w[\cos(w^T(x - x'))]$

We are given

$$k_p(x, x') = \mathbb{E}_w \left[e^{iw^T(x-x')} \right].$$

Using Euler's formula, $e^{iy} = \cos(y) + i\sin(y)$, we can write

$$\begin{aligned} k_p(x, x') &= \mathbb{E}_w \left[\cos(w^T(x - x')) + i \sin(w^T(x - x')) \right] \\ &= \mathbb{E}_w \left[\cos(w^T(x - x')) \right] \\ &\quad + i\mathbb{E}_w \left[\sin(w^T(x - x')) \right]. \end{aligned}$$

Since w follows a zero-mean Gaussian distribution, its probability density is symmetric about the origin: $p(w) = p(-w)$. Furthermore, the sine function is odd, so

$$\sin((-w)^T(x - x')) = -\sin(w^T(x - x')).$$

Therefore, the positive and negative contributions cancel, giving

$$\mathbb{E}_w \left[\sin(w^T(x - x')) \right] = 0.$$

Consequently,

$$\boxed{k_p(x, x') = \mathbb{E}_w \left[\cos(w^T(x - x')) \right].}$$

ii : Show that $\mathbb{E}_{w,b}[z_w(x)z_w(x')] = k_p(x, x')$

We define

$$z_w(x) = \sqrt{2}\cos(w^T x + b),$$

where $w \sim p(w)$ and $b \sim \text{Uniform}(0, 2\pi)$.

The product of two random features is

$$z_w(x)z_w(x') = 2\cos(w^T x + b)\cos(w^T x' + b).$$

Using the identity

$$2\cos(\alpha)\cos(\beta) = \cos(\alpha + \beta) + \cos(\alpha - \beta),$$

we obtain

$$\begin{aligned} z_w(x)z_w(x') &= \cos\left(w^T(x + x') + 2b\right) \\ &\quad + \cos\left(w^T(x - x')\right). \end{aligned}$$

Taking the expectation with respect to b, the first term vanishes because b is uniformly distributed over $[0, 2\pi)$:

$$\begin{aligned} &\mathbb{E}_b \left[\cos\left(w^T(x + x') + 2b\right) \right] \\ &= \frac{1}{2\pi} \int_0^{2\pi} \cos\left(w^T(x + x') + 2b\right) \, db \\ &= 0. \end{aligned}$$

Thus,

$$\mathbb{E}_b[z_w(x)z_w(x')] = \cos(w^T(x - x')).$$

Taking the expectation with respect to w and applying part (i),

$$\begin{aligned} \mathbb{E}_{w,b}[z_w(x)z_w(x')] &= \mathbb{E}_w \left[\mathbb{E}_b[z_w(x)z_w(x')] \right] \\ &= \mathbb{E}_w \left[\cos(w^T(x - x')) \right] \\ &= k_p(x, x'). \end{aligned}$$

Therefore,

$$\boxed{\mathbb{E}_{w,b}[z_w(x)z_w(x')] = k_p(x,x').}$$

iii : Show that $\mathbb{E}[z(x)^T z(x')] = k_p(x,x')$

Define the D-dimensional random Fourier feature mapping

$$z(x) = \frac{1}{\sqrt{D}} \begin{bmatrix} z_{w_1}(x) \\ z_{w_2}(x) \\ \vdots \\ z_{w_D}(x) \end{bmatrix},$$

where the random parameters (w_j, b_j) are independently sampled for $j = 1, \ldots, D$.

The inner product between the feature vectors is

$$z(x)^T z(x') = \frac{1}{D} \sum_{j=1}^{D} z_{w_j}(x) z_{w_j}(x').$$

By linearity of expectation,

$$\begin{aligned} \mathbb{E}[z(x)^T z(x')] &= \mathbb{E}\left[\frac{1}{D} \sum_{j=1}^{D} z_{w_j}(x) z_{w_j}(x')\right] \\ &= \frac{1}{D} \sum_{j=1}^{D} \mathbb{E}[z_{w_j}(x) z_{w_j}(x')]. \end{aligned}$$

From part (ii), each expected product equals $k_p(x,x')$. Therefore,

$$\begin{aligned} \mathbb{E}[z(x)^T z(x')] &= \frac{1}{D} \sum_{j=1}^{D} k_p(x,x') \\ &= \frac{D}{D} k_p(x,x') \\ &= k_p(x,x'). \end{aligned}$$

Hence,

$$\boxed{\mathbb{E}[z(x)^T z(x')] = k_p(x,x').}$$

This proves that the random Fourier feature mapping provides an unbiased estimate of the kernel function.

b. *[5 points]* **Random Fourier Features and the RBF Kernel**. As mentioned in part (a), using different distributions $p(\mathbf{w})$ induces different valid kernels. Using the $p(\mathbf{w})$ given in part (a), show that expected value of the inner product between random Fourier features is the RBF kernel i.e.

$$E_{\mathbf{w}}[\mathbf{z}(\mathbf{x})^T \mathbf{z}(\mathbf{x}')] = \exp\left(-\frac{||\mathbf{x} - \mathbf{x}'||_2^2}{2\gamma^2}\right)$$

Hint: The PDF for a variable $X \in \mathbb{R}^d$ following normal distribution with mean μ and covariance matrix Σ is as follows:

$$P(X = x) = ((2\pi)^d |\Sigma|)^{-1/2} \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right)$$

where $|\Sigma| = \det(\Sigma)$ denote the determinant of matrix Σ. In addition, if $\Sigma = \text{diag}(\sigma^2, \ldots, \sigma^2)$, then $|\Sigma| = \sigma^{2d}$, and $\Sigma^{-1} = \text{diag}(\sigma^{-2}, \ldots, \sigma^{-2})$.

From part (a), we established that

$$\mathbb{E}_{w,b}[z(x)^T z(x')] = \mathbb{E}_w[\cos(w^T(x - x'))].$$

We want to show that this expectation is an RBF kernel. Let $u = x - x'$. Since w follows a zero-mean Gaussian distribution with covariance matrix

$$\Sigma = \frac{1}{\gamma^2} I_d,$$

its probability density function is

$$p(w) = \left(\frac{\gamma^2}{2\pi}\right)^{d/2} \exp\left(-\frac{\gamma^2}{2}\|w\|_2^2\right).$$

Using the complex exponential and taking the real part, we obtain

$$\mathbb{E}_w[\cos(w^T u)] = \text{Re}\left(\mathbb{E}_w[e^{iw^T u}]\right).$$

The characteristic function of a multivariate Gaussian distribution with mean zero and covariance Σ is

$$\mathbb{E}_w[e^{iw^T u}] = \exp\left(-\frac{1}{2}u^T \Sigma u\right).$$

Substituting $\Sigma = \frac{1}{\gamma^2} I_d$ gives

$$\begin{aligned}\mathbb{E}_w[e^{iw^T u}] &= \exp\left(-\frac{1}{2\gamma^2}u^T u\right) \\ &= \exp\left(-\frac{\|u\|_2^2}{2\gamma^2}\right).\end{aligned}$$

Since this expression is real, taking its real part does not change it. Therefore,

$$\mathbb{E}_w[\cos(w^T u)] = \exp\left(-\frac{\|u\|_2^2}{2\gamma^2}\right).$$

Finally, substituting $u = x - x'$, we obtain

$$\boxed{\mathbb{E}_{w,b}[z(x)^T z(x')] = \exp\left(-\frac{\|x - x'\|_2^2}{2\gamma^2}\right).}$$

Thus, the expected inner product of the random Fourier features is an RBF kernel.

Note: This derivation follows the Gaussian density specified in part (a). If the intended RBF kernel is instead

$$k(x, x') = \exp\left(-\frac{\gamma^2}{2}\|x - x'\|_2^2\right),$$

then the Gaussian distribution must have covariance $\Sigma = \gamma^2 I_d$, rather than $\Sigma = \frac{1}{\gamma^2} I_d$.

c. *[5 points]* **Concentration Bounds** In part (a) we derived our function $\mathbf{z}$ which serve as a good approximation to the kernel function. Our results let us get an upper bound our approximation error for the kernel function. Explain why we can apply Hoeffding's inequality to obtain

$$p(|\mathbf{z}(\mathbf{x})^T\mathbf{z}(\mathbf{x}') - k(\mathbf{x},\mathbf{x}')| \geq \epsilon) \leq 2\exp\left(-D\epsilon^2/8\right)$$

We want to show that Hoeffding's inequality gives

$$\Pr\left(\left|z(x)^T z(x') - k(x,x')\right| \geq \epsilon\right) \leq 2\exp\left(-\frac{D\epsilon^2}{8}\right).$$

From part (a), the random Fourier feature mapping is defined as

$$z(x) = \frac{1}{\sqrt{D}}\begin{bmatrix} z_{w_1}(x) \\ \vdots \\ z_{w_D}(x) \end{bmatrix}, \qquad z_{w_j}(x) = \sqrt{2}\cos(w_j^T x + b_j).$$

Therefore, the inner product can be written as

$$z(x)^T z(x') = \frac{1}{D}\sum_{j=1}^{D} z_{w_j}(x) z_{w_j}(x').$$

Define

$$X_j = z_{w_j}(x) z_{w_j}(x').$$

Then

$$z(x)^T z(x') = \frac{1}{D}\sum_{j=1}^{D} X_j.$$

From part (a), each random variable satisfies

$$\mathbb{E}[X_j] = k(x,x').$$

Furthermore, because the random parameters (w_j, b_j) are independently sampled, the random variables $X_1, \ldots, X_D$ are independent.

Since $|\cos(\cdot)| \leq 1$, we have

$$|z_{w_j}(x)| \leq \sqrt{2}, \qquad |z_{w_j}(x')| \leq \sqrt{2}.$$

Consequently,

$$|X_j| = |z_{w_j}(x) z_{w_j}(x')| \leq 2,$$

which implies

$$-2 \leq X_j \leq 2.$$

Hoeffding's inequality states that for independent random variables $X_j \in [a_j, b_j]$,

$$\Pr\left(\left|\frac{1}{D}\sum_{j=1}^{D}(X_j - \mathbb{E}[X_j])\right| \geq \epsilon\right) \leq 2\exp\left(-\frac{2D^2\epsilon^2}{\sum_{j=1}^{D}(b_j - a_j)^2}\right).$$

In our case, $a_j = -2$ and $b_j = 2$, so

$$\sum_{j=1}^{D}(b_j - a_j)^2 = \sum_{j=1}^{D}(2-(-2))^2 = 16D.$$

Substituting this into Hoeffding's inequality yields

$$\begin{aligned}\Pr\left(\left|\frac{1}{D}\sum_{j=1}^{D}(X_j - \mathbb{E}[X_j])\right| \geq \epsilon\right) &\leq 2\exp\left(-\frac{2D^2\epsilon^2}{16D}\right) \\ &= 2\exp\left(-\frac{D\epsilon^2}{8}\right).\end{aligned}$$

Finally, since

$$\frac{1}{D}\sum_{j=1}^{D}\mathbb{E}[X_j] = k(x,x'),$$

we conclude that

$$\boxed{\Pr\left(\left|z(x)^T z(x') - k(x,x')\right| \geq \epsilon\right) \leq 2\exp\left(-\frac{D\epsilon^2}{8}\right).}$$

Thus, Hoeffding's inequality applies because the random feature products are independent, bounded random variables whose average is an unbiased estimator of the kernel. The bound also shows that increasing the number of random features D exponentially decreases the probability of an approximation error greater than or equal to ϵ.

Administrative

A6.

a. *[2 points]* About how many hours did you spend on this homework? There is no right or wrong answer :)

I spent about 20 hours on this homework.